

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf\_2ae09dfb-e3e\agent-a9631dddeee859578.jsonl`
- SHA-256: `728f78f83a7680c51445199eed2f1406e657c8dc915813b298fe8e05da1106d6`
- Source modified: `2026-06-13T19:56:53+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf\_2ae09dfb-e3e`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:55:36.017Z)

Design P4: wire the EXISTING audio hit detector as a cue/feature (NOT a special-case). Read backend/pipeline/audio/hit_detector.py (extract_audio, onset_envelope, detect_hits, HitEvent, detect_hits_in_video) and the audio-e1 history (docs/archives/research/AUDIO_RALLY_DETECTION_PLAN.md if present). Design a producer that emits per-window audio features (e.g. hit_count, hit_rate, mean_strength) into the candidate stats / harness feature path, default-OFF (a cue_flag), so the learned scorer can weight it ? honest expectation: prior E1 work found audio weak (~2.2x discrimination), so this is "add the signal, let the harness/golden-set decide", not "adopt audio". Specify where it plugs into FusionSegmenter (another enabled-cue branch in _enrich_candidate_stats) and the feature names. Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. Read the ACTUAL files, quote file:line. The fusion P2 segmenter (FusionSegmenter), fusion_features.py, and PersonDetector.detections_per_frame are ALREADY on master. The experiment harness (backend/eval/experiment.py: Variant/compare/compare_unlabeled/load_video_candidates) is shipped. Be precise.

2. assistant (2026-06-13T19:55:38.301Z)

I'll help you design the wiring of the existing audio hit detector as a cue/feature. Let me start by reading the relevant files to understand the current implementation.

3. user (2026-06-13T19:55:38.543Z)

```
1      """Audio shuttle-hit ("thwack") detection ? local, dependency-light (numpy + ffmpeg).
2
3      Stages 0?1 of docs/AUDIO_RALLY_DETECTION_PLAN.md: extract the audio track, build a
4      spectral-flux onset envelope (with pre-emphasis to boost the broadband HF transient
5      a racket?shuttle contact makes), threshold it adaptively, and return (timestamp,
6      strength) hit candidates. Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess)
7      only for extraction. Consumed by the E1 go/no-go probe; later feeds rally fusion
8      (ADR-007) if E1 passes. WASB is never modified ? this is a separate signal.
9      """
10     from __future__ import annotations
11
12     import os
13     import wave
14     import tempfile
15     import subprocess
16     from dataclasses import dataclass
17     from typing import List, Optional, Tuple
18
19     import numpy as np
20
21
22     @dataclass
```

```

23     class HitEvent:
24         t: float          # seconds
25         strength: float   # onset-envelope peak value (arbitrary units)
26
27
28     def extract_audio(video_path: str, out_wav: Optional[str] = None, sr: int = 16000) ->
    str:
29         """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
30         Audio-only decode, so it's fast even for multi-GB video."""
31         if out_wav is None:
32             fd, out_wav = tempfile.mkstemp(suffix=".wav")
33             os.close(fd)
34             cmd = ["ffmpeg", "-y", "-i", video_path, "-vn", "-ac", "1", "-ar", str(sr),
35                 "-c:a", "pcm_s16le", out_wav]
36             subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL)
37             return out_wav
38
39
40     def load_wav(path: str) -> Tuple[np.ndarray, int]:
41         """Read a PCM WAV ? (float32 samples in [-1,1], sample_rate). Mono-downmixes."""
42         with wave.open(path, "rb") as w:
43             sr, n, sw, ch = w.getframerate(), w.getnframes(), w.getsampwidth(),
    w.getnchannels()
44             raw = w.readframes(n)
45             if sw == 2:
46                 a = np.frombuffer(raw, dtype=np.int16).astype(np.float32) / 32768.0
47             elif sw == 4:
48                 a = np.frombuffer(raw, dtype=np.int32).astype(np.float32) / 2147483648.0
49             elif sw == 1:
50                 a = np.frombuffer(raw, dtype=np.uint8).astype(np.float32) / 128.0 - 1.0
51             else:
52                 raise ValueError(f"unsupported sample width {sw}")
53             if ch > 1:
54                 a = a.reshape(-1, ch).mean(axis=1)
55             return a, sr
56
57
58     def _preemphasis(x: np.ndarray, coef: float = 0.97) -> np.ndarray:
59         if len(x) < 2:
60             return x.copy()
61         return np.append(x[0], x[1:] - coef * x[:-1])
62
63
64     def onset_envelope(samples: np.ndarray, sr: int, frame: int = 1024,
65         hop: int = 256, band: Optional[Tuple[float, float]] = None) ->
    Tuple[np.ndarray, float]:
66         """Spectral-flux onset strength per hop (pure-numpy STFT). Returns (env,
    hop_seconds).
67
68         `band=(lo_hz, hi_hz)` restricts the flux to a frequency band ? a shuttle "thwack"
69         is HF-rich while voices/footsteps live lower, so band-limiting (e.g. 3-9 kHz)
70         suppresses non-shuttle transients. Needs sr high enough for hi_hz (Nyquist =
    sr/2)."""
71         x = _preemphasis(np.asarray(samples, dtype=np.float32))
72         hop_sec = hop / float(sr)
73         if len(x) < frame:
74             return np.zeros(0, dtype=np.float32), hop_sec
75         win = np.hanning(frame).astype(np.float32)
76         n_frames = 1 + (len(x) - frame) // hop
77         idx = np.arange(frame)[None, :] + hop * np.arange(n_frames)[:, None]
78         frames = x[idx] * win
79         mag = np.abs(np.fft.rfft(frames, axis=1))
80         if band is not None:
81             freqs = np.fft.rfftfreq(frame, d=1.0 / sr)

```

```

82         keep = (freqs >= band[0]) & (freqs <= band[1])
83         if keep.any():
84             mag = mag[:, keep] # band-limited flux
85             diff = np.diff(mag, axis=0) # frame-to-frame change
86             flux = np.sqrt((np.maximum(diff, 0.0) ** 2).sum(axis=1)) # positive (onset) flux
87             env = np.concatenate([[0.0], flux]).astype(np.float32) # align to n_frames
88             return env, hop_sec
89
90
91 def _moving_stats(env: np.ndarray, win: int) -> Tuple[np.ndarray, np.ndarray]:
92     """Sliding-window mean and std (reflect-padded, via cumsum). win forced odd."""
93     if win % 2 == 0:
94         win += 1
95     pad = win // 2
96     p = np.pad(env.astype(np.float64), pad, mode="reflect")
97     c1 = np.cumsum(np.insert(p, 0, 0.0))
98     c2 = np.cumsum(np.insert(p * p, 0, 0.0))
99     s1 = c1[win:] - c1[:-win]
100    s2 = c2[win:] - c2[:-win]
101    mean = s1 / win
102    var = np.maximum(s2 / win - mean * mean, 0.0)
103    return mean[:len(env)], np.sqrt(var)[:len(env)]
104
105
106 def detect_hits(samples: np.ndarray, sr: int, k: float = 2.5, win_frames: int = 41,
107                min_gap_sec: float = 0.08, band: Optional[Tuple[float, float]] = None)
-> List[HitEvent]:
108     """Detect impact transients: local maxima of the onset envelope that exceed an
109     adaptive threshold (moving mean + k*std), keeping the strongest within
110     `min_gap_sec`.
111     `k` is the sensitivity knob (lower = more hits); `band` band-limits the onset flux
112     to the shuttle-click frequencies (e.g. (3000, 9000))."""
113     env, hop_sec = onset_envelope(samples, sr, band=band)
114     if len(env) < 3:
115         return []
116     mean, std = _moving_stats(env, win_frames)
117     thr = mean + k * std
118     is_peak = (env[1:-1] >= env[:-2]) & (env[1:-1] > env[2:]) & (env[1:-1] > thr[1:-1])
119     peak_idx = np.nonzero(is_peak)[0] + 1
120     if peak_idx.size == 0:
121         return []
122     # enforce min inter-peak gap: greedily keep strongest first
123     min_gap = max(1, int(round(min_gap_sec / hop_sec)))
124     order = peak_idx[np.argsort(-env[peak_idx])]
125     taken = np.zeros(len(env), dtype=bool)
126     kept = []
127     for i in order:
128         lo, hi = max(0, i - min_gap), min(len(env), i + min_gap + 1)
129         if taken[lo:hi].any():
130             continue
131         taken[i] = True
132         kept.append(i)
133     kept.sort()
134     return [HitEvent(t=float(i * hop_sec), strength=float(env[i])) for i in kept]
135
136
137 def detect_hits_in_video(video_path: str, sr: int = 16000, **kw) -> List[HitEvent]:
138     """Convenience: extract audio from a video and detect hits. Cleans up the temp
139     wav."""
140     wav = extract_audio(video_path, sr=sr)
141     try:
142         samples, real_sr = load_wav(wav)
143         return detect_hits(samples, real_sr, **kw)
144     finally:
145         try:

```

```
144         os.remove(wav)
145     except OSError:
146         pass
147
```

4. user (2026-06-13T19:55:38.761Z)

```
1     # Audio-based rally detection ? detailed plan
2
3     **Status:** Accepted direction (ADR-007) · experiments gated on golden eval labels ·
**Date:** 2026-06-07
4     **Decision in one line:** add a fully-local **audio "thwack" hit-detection cue, fused
with the WASB shuttle
5     trajectory**, as the next rally-detection lever ? because it directly attacks our recall
bottleneck using an
6     input we currently throw away. Pose/stance detection is the parked #2 (see ADR-007 /
[research](RALLY_DETECTION_RESEARCH.md)).
7
8     ## 1. Why audio (grounded in the research)
9     - **#1 cheap local lever** in the verified survey (`docs/RALLY_DETECTION_RESEARCH.md`).
Audio+visual **fusion
10     lifts rally RECALL** ? tennis HMM: 83% fused vs 54% visual-only / 63% audio-only at
~unchanged precision ?
11     and recall is exactly where WASB-derived windows fail us (IoU-0.5 candidate recall
ceilinged at ~0.43 on
12     TestLargeVideo).
13     - **Fully local, permissive, CPU-light** (classical MFCC/energy methods; no restrictive
weights) ? keeps the
14     privacy lever and adds ~zero marginal cost.
15     - **Uses an input we currently ignore.** Our GoPro clips already carry audio.
16
17     ## 2. What we detect, and the rally model
18     - **Hit event** = racket?shuttle contact ("thwack"), a short broadband transient.
19     - **Rally** = a temporally-clustered run of hits with rhythmic cadence (a shot roughly
every ~0.4?2 s,
20     alternating ends). **Dead time** = no/sparse hits. A **single isolated hit** = the
between-point service
21     feed/toss ? i.e. exactly the false-fire pattern we've been fighting. So hit
*clusters*, not single hits,
22     define rallies.
23
24     ## 3. The GoPro-audio reality (honest, and the key risk)
25     All clips provided so far have audio (GoPro records it by default). BUT: a shuttle
"thwack" is **much quieter**
26     than a tennis/table-tennis impact (lighter projectile), so the **SNR transfer is the
dominant open risk** ? the
27     research's audio numbers come from louder sports. Gym reverb, wind, and crowd add noise.
**Mitigations:**
28     1. **E1 (below) is an explicit GO/NO-GO probe on the real GoPro audio** before we build
anything heavy.
29     2. Make **"record with audio enabled, mic unobstructed"** a capture requirement (added
to
30     `docs/VIDEO_RECORDING_GUIDELINES.md`).
31
32     ## 4. Architecture (all local, CPU-light)
33     - **Stage 0 ? Extract:** `ffmpeg` ? mono WAV (~16?22 kHz; keep HF ? clicks are
broadband). Cached per video,
34     keyed like the frame/trajectory caches.
35     - **Stage 1 ? Onset/peak candidates (NO training):** band-emphasis/high-pass ? short-
time energy + spectral
36     flux ? adaptive (EMA/median) threshold ? peak-pick ? `(timestamp, strength)` candidate
impacts. (Research E1.)
37     - **Stage 2 ? Hit confidence / FP suppression:** a likelihood-ratio confidence
(research: raw peaks ~41?50 F ?
38     ~68?77 F) and optionally a tiny MFCC-window classifier (GMM/logreg, hit-vs-not),
```

```

bootstrapped from the
39     user's golden rally spans (hits inside rallies = positives). (Research E2.)
40     - **Stage 3 ? Hits ? rally structure:** cluster impacts by inter-hit gap; a cluster with
?K hits and plausible
41     cadence = a rally; **START ? first hit (the serve), END ? last hit + a short tail**;
drop isolated single
42     hits (the feed).
43     - **Stage 4 ? Fuse with the WASB trajectory** (in OUR rally layer ? WASB stays a frozen
black box; we do NOT
44     feed audio into WASB. See ADR-007 "Integration architecture" for the full rationale +
the Option-B end-state):
45     - **Recall recovery (the main win):** propose a candidate rally wherever there's a
dense hit-cluster but WASB
46     proposed nothing.
47     - **Precision / features:** add per-candidate audio features ? `hit_count`,
`hit_rate`, inter-hit-interval
48     mean/std (cadence regularity), `time_since_last_hit`, end-of-window energy ? to the
existing
49     `distill_local` student (Research E3).
50     - **Joint rule:** a rally needs shuttle-motion **and** a hit-cluster (precision), OR
an audio cluster
51     recovers a WASB miss (recall).
52     - **Stage 5 ? Eval:** vs **golden labels** (the rater CSVs from the VLC annotator) at
IoU 0.5, leave-one-video-
53     out; report recall lift + precision vs the WASB-only baseline and the trajectory-only
distilled model.
54
55     ## 5. Experiments (sequenced ? E1 is the gate)
56     | # | Experiment | Needs | Decides |
57     |---|---|---|---|
58     | **E1** | Extract audio ? energy-peak hits ? overlay hit-density on WASB windows
**and** golden labels | audio + ?1 golden-rated video | **GO/NO-GO** : do hit-clusters align with
real rallies + recover WASB-missed ones on *real GoPro* audio? |
59     | **E2** | Add likelihood-ratio / MFCC confidence to suppress false peaks | E1 + a few
labeled hits | F-score lift of the hit stream |
60     | **E3** | Add audio features to `distill_local`; re-eval IoU-0.5 LOVO | E1/E2 + ?2
golden videos | does fused beat WASB-only + trajectory-only distilled? |
61     | **E4** | Use first/last hit to tighten rally start/end | E1 | boundary-IoU improvement
|
62
63     ## 6. Code & dependencies
64     - New module `backend/pipeline/audio/hit_detector.py` (extract + onset/peak +
confidence). Pure DSP helpers
65     unit-tested on **synthetic click audio** (no fixtures); ffmpeg via subprocess (already
a dep).
66     - Reuse: feed audio features into `backend/eval/distill_local.py` (pluggable
`feature_fn`); score with the
67     existing `metrics.py` / golden loaders.
68     - **Deps:** **numpy + stdlib `wave` (no scipy needed)** for filtering/STFT ? the
spectral-flux onset is a pure
69     numpy STFT, so the originally-anticipated scipy dependency was not added. Avoid
heavy/AGPL audio stacks. Any
70     later small CNN = trained in-house (no restrictive weights).
71     - **Licensing:** all permissive ? no new model weights for Stage 1. Consistent with
ADR-005.
72
73     ## 7. Risks & mitigations
74     - **Shuttle SNR (top risk):** E1 is the empirical gate; if NO-GO, audio is deferred, not
forced.
75     - **No crowd applause** in amateur single-camera clips (a cue the literature leans on
for end-of-point) ? rely
76     on **hit-cadence cessation**, not applause.
77     - **Mic occlusion / wind** ? recording guideline (audio on, unobstructed, avoid
aggressive wind-suppression).
78     - **A/V sync:** audio and frames share the same file timeline ? no sync work needed.

```

```

79
80     ## 8. Gating & sequencing
81     - All measurement is **gated on golden labels** (user rating via the VLC annotator ?
`import-local` / the
82     in-repo `--labels` loaders). So: rate a few videos ? **E1 go/no-go** ? E2/E3.
83     - Roadmap order: **AUDIO (now)** ? **[a TBD owner hunch ? slot reserved]* ? **POSE/STANCE
heuristic (#2, parked;
84     expected substantially harder)** ? **OPTION B: an OWNED end-to-end multimodal rally
model** (frames + audio +
85     pose + ? ? rallies), fine-tuned on our golden corpus instead of depending on WASB's
pretrained weights ? the
86     intended destination once the corpus is healthy. The modular cues here are how we earn
it (each one cheap,
87     debuggable, and a generator of the labels Option B needs). Full rationale: ADR-007
"Integration architecture".
88
89     ## 9. Success criteria
90     - **E1:** audio hit-clusters recover a meaningful share of WASB-missed golden rallies on
real GoPro audio
91     (recall lift), with hit-clusters aligning to true rallies. Pass ? proceed; fail ?
defer audio, revisit mic/setup.
92     - **E3:** the fused model's held-out (LOVO) F1 beats both the WASB-only baseline and the
trajectory-only
93     distilled student.
94
95     ## 10. E1 RESULTS ? preliminary, on existing GoPro audio vs Gemini SILVER labels
(2026-06-07)
96     Ran the E1 probe (`backend/eval/audio/e1_probe.py`) + a band*k sweep on Adarsh (36
rallies) and
97     TestLargeVideo (7). **Verdict: AMBER ? leaning negative for the *classical* detector.**
98     - ? **Recall signal is real:** the detector fires in **100% of rallies** and **100% of
WASB-missed rallies
99     contain hit-clusters** ? audio *does* light up where rallies are.
100    - ?? **Precision/discrimination ceilings ~2.2** (in-rally vs *mid-match* dead-time hit
density), on BOTH videos.
101    Audio-only proposal F1 (0.13?0.27) trails WASB-only.
102    - ? **Band-pass did NOT help** (tested low/mid/high bands x k at 32 kHz): HF "shuttle-
click" bands gave *lower*
103    discrimination, not higher. Shuttle hits have no band-separable signature in this
audio.
104    - ? **Not a warm-up/label artifact:** mid-match dead time (between rallies) still has
~0.6 hits/s vs ~1.3
105    in-rally ? the ~2.2x ceiling holds even excluding pre/post-match regions. Root cause:
between-point activity
106    (the loser tossing the shuttle back ? our very false-fire ? plus footwork/talk) makes
transients a classical
107    onset detector can't distinguish from rally hits.
108    - `k` (threshold) is the only real lever and trades coverage for discrimination (~3x
costs half the rallies).
109    - A `band` knob was added to `onset_envelope`/`detect_hits` and KEPT (cheap, optional) ?
but it is **not** the fix.
110
111    **Implication / next:** the cheap classical-onset cue is **too weak to be a clean
precision signal**, and a
112    band-pass won't rescue it. Remaining audio paths, in order of honesty:
113    1. **E2 ? a learned TIMBRE classifier** (MFCC/log-mel ? shuttle-thwack vs
toss/footstep/voice). This separates by
114    *timbre*, which a band-pass/onset cannot ? the only plausible way to break the ~2.2x
ceiling. **Needs labeled
115    hits** (bootstrap from human-gold rally spans, or hand-label a few minutes). Real
work.
116    2. **Re-confirm E1 on HUMAN-GOLD labels** ? cheap, but the warm-up check already
suggests it won't move the
117    ceiling much (dead-time activity is mid-match, not just warm-up).
118    3. **Re-prioritize:** given the classical ceiling, consider the owner's "other hunch" /

```

the pose-stance #2 /
119 leaning on more golden data for the WASB+distilled path, rather than over-investing
in classical audio.
120

5. user (2026-06-13T19:55:39.092Z)

backend\pipeline\detectors\fusion_features.py

6. user (2026-06-13T19:55:39.170Z)

No files found

7. assistant (2026-06-13T19:55:40.988Z)

Let me find the FusionSegmenter and fusion_features files:

8. user (2026-06-13T19:55:41.519Z)

No files found

9. user (2026-06-13T19:55:41.698Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of scale/translation-invariant
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12     - `per_frame` is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13         index (so it aligns directly with the shuttle trajectory's `frame` ? no clip re-
timing).
14     - A shuttle point has `frame`, `visible`, `x`, `y` (pixels) ? the WASB
trajectory shape.
15     - `court_polygon` is normalised 0-1 `[(x, y), ...]` or `None` (None ? no mask,
every
16         detection counts ? the player-count-only mode).
17     - `net` is `(axis 'x'|'y', position 0-1 normalised)` or `None` (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except `mean_player_motion` (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
```

```

34         return False
35     x, y = point
36     inside = False
37     n = len(polygon)
38     j = n - 1
39     for i in range(n):
40         xi, yi = polygon[i]
41         xj, yj = polygon[j]
42         # Does the horizontal ray at y cross edge (i, j)?
43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47         j = i
48     return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71         point meaningless, so a masked check returns False rather than testing a bogus
72         (0,0)."""
73         if court_polygon is None:
74             return True
75         if frame_width <= 0 or frame_height <= 0:
76             return False
77         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                       frame_height: float,
81                       court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82         """Per-frame count of in-court persons (one entry per sampled frame)."""
83         return [
84             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
85             court_polygon))
86             for _frame_idx, dets in sorted(per_frame.items())
87         ]
88
89     def _median(vals: Sequence[float]) -> float:
90         if not vals:
91             return 0.0
92         s = sorted(vals)
93         n = len(s)

```



```

94         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97     def players_in_court(counts: Sequence[int]) -> float:
98         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99         return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
104         flicker)."""
105         if not counts:
106             return 0.0
107         return sum(1 for c in counts if c >= min_players) / len(counts)
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                           frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112         normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113         not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:
115             return 0.0
116         frames = sorted(per_frame.keys())
117         steps: List[float] = []
118         for a, b in zip(frames, frames[1:]):
119             prev = {tid: box for tid, box in per_frame[a]}
120             for tid, box in per_frame[b]:
121                 if tid in prev:
122                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125         if not steps:
126             return 0.0
127         return sum(steps) / len(steps)
128
129
130     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                         frame_height: float, net: Optional[Net],
132                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137
138         """
139         if net is None or not per_frame:
140             return 0.0
141         axis, pos = net
142         both = 0
143         for _frame_idx, dets in per_frame.items():
144             near, far = False, False
145             for _tid, box in dets:
146                 if not person_in_court(box, frame_width, frame_height, court_polygon):
147                     continue
148                 fx, fy = _foot_norm(box, frame_width, frame_height)
149                 coord = fx if axis == "x" else fy
150                 if coord < pos:
151                     near = True
152             else:
153                 far = True
154             if near and far:
155                 both += 1
156         return both / len(per_frame)

```

```

157
158     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                        pad_frac: float) -> bool:
160         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161         width/height (the 'adjacent / in-hand' tolerance)?"""
162         x1, y1, x2, y2 = box
163         px = pad_frac * frame_width
164         py = pad_frac * frame_height
165         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                                frame_width: float, frame_height: float,
170                                pad_frac: float = 0.03) -> float:
171         """Fraction of (visible shuttle frames that were also person-sampled) in which the
172         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
173         ~0**
174         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
175         """
176         if not per_frame or frame_width <= 0 or frame_height <= 0:
177             return 0.0
178         aligned = 0
179         collocated = 0
180         for p in shuttle_points or []:
181             if not getattr(p, "visible", False):
182                 continue
183             dets = per_frame.get(int(p.frame))
184             if dets is None:
185                 # shuttle frame wasn't person-sampled ? can't judge
186                 continue
187             aligned += 1
188             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
189                    for _tid, box in dets):
190                 collocated += 1
191         return collocated / aligned if aligned else 0.0
192
193     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
194     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
195     PERSON_FEATURE_NAMES = [
196         "players_in_court", "two_sided_motion", "mean_player_motion",
197         "in_court_ratio", "shuttle_player_colocation",
198     ]
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                                frame_width: float, frame_height: float,
202                                court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203                                net: Optional[Net] = None,
204                                min_players: int = 2) -> Dict[str, float]:
205         """Aggregate the per-window person features from per-frame detections + the in-
206         window
207         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
208         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
209         return {
210             "players_in_court": players_in_court(counts),
211             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
212             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
213             "in_court_ratio": in_court_ratio(counts, min_players),
214             "shuttle_player_colocation": shuttle_player_colocation(
per_frame, shuttle_points, frame_width, frame_height),

```

215 }
216

10. assistant (2026-06-13T19:55:43.063Z)

Now let me find the actual FusionSegmenter file:

11. user (2026-06-13T19:55:43.383Z)

No files found

12. user (2026-06-13T19:55:43.449Z)

backend\pipeline\detectors\fusion_features.py
backend\pipeline\segmenters\fusion_hybrid.py

13. user (2026-06-13T19:55:44.895Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
13     the
14     torchvision/supervision model ? so the default (person-OFF) path never loads a
15     detector
16     model nor touches the GPU. (torch/cv2 themselves are imported by the registry
17     regardless,
18     via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
19     - ``_select_for_handoff`` ? optional served Gate A/B: when
20     ``indexing.fusion.min_crossings``
21     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
22     are
23     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
24     **Persisted candidates remain the full superset** regardless, so the offline
25     experiment
26     harness can re-score any variant.
27
28     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
29     but
30     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
31     seeds
32     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
33     default
34     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
35     through the
36     experiment harness (EXPERIMENT_HARNESS.md), never silently.
37     """
38
39     from typing import Any, Dict, List, Optional, Tuple
40
41     from backend.pipeline.segmenters.base import segmenter_registry
42     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
43     from backend.pipeline.detectors.base import DetectorRunner
44     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
45     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
46     from backend.pipeline.detectors import rally_gate
47     from backend.pipeline.detectors import fusion_features as ff
```

```

37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
41 B)."""
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48             self._person: Optional[Any] = None
49             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50             # over the whole trajectory once per candidate window (it depends only on
`points`).
51             self._axis_cache_key: Optional[int] = None
52             self._axis_cache: Optional[tuple] = None
53
54         @property
55         def name(self) -> str:
56             return "fusion_hybrid"
57
58         @property
59         def description(self) -> str:
60             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                     "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                     "AI provider sets semantic boundaries.")
63
64         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66             if substrate == "tracknet":
67                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                 return TrackNetRunner(cfg), {
69                     "weights_path": cfg.weights_path,
70                     "queue_length": cfg.queue_length,
71                     "fusion_substrate": "tracknet",
72                 }
73             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                     frame_width: float, video_path: str,
78                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82             # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83             gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,
84                                                  min_crossings=0,
estimate=self._axis_estimate(points))
85             stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87             fcfg = idx_cfg.get("fusion", {})
88             if fcfg.get("cue_flags", {}).get("person", False):

```

```

89         stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90     return stats
91
92     def _axis_estimate(self, points: List[Any]) -> tuple:
93         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94         computed once per video, not once per candidate window (it depends only on
points)."""
95         key = id(points)
96         if self._axis_cache_key != key or self._axis_cache is None:
97             self._axis_cache_key = key
98             self._axis_cache = rally_gate.estimate_play_axis(points)
99         est = self._axis_cache
100         assert est is not None
101         return est
102
103     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                         frame_width: float, video_path: str,
105                         fcfg: Dict[str, Any]) -> Dict[str, float]:
106         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107         frame range and compute the scale/translation-invariant person features. Lazily
builds
108         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110         from backend.pipeline.detectors.person_detector import PersonDetector
111         if self._person is None:
112             self._person = PersonDetector(self.config)
113
114         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119         fw = det["frame_width"] or frame_width
120         fh = det["frame_height"]
121         # If we can't trust the frame dims, every normalised feature would silently
collapse
122         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123         if fw <= 0 or fh <= 0:
124             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126         poly = fcfg.get("court_polygon")
127         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                         court_polygon=polygon, net=net)
132
133     def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                           window_stats: List[Dict[str, Any]],
135                           idx_cfg: Dict[str, Any]) -> List[int]:
136         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137         (0) keep every window ? identical to the substrate. Persisted candidates are
untouched (full superset)."""
138         fcfg = idx_cfg.get("fusion", {})
139         min_crossings = int(fcfg.get("min_crossings", 0))
140         min_players = int(fcfg.get("min_players", 0))
141         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
142

```

```

143
144         keep: List[int] = []
145         for i, st in enumerate(window_stats):
146             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                 continue
148             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                 continue
150                 keep.append(i)
151         return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155

```

14. user (2026-06-13T19:55:45.231Z)

No files found

15. user (2026-06-13T19:55:45.296Z)

backend\eval\distill_local.py

16. user (2026-06-13T19:55:47.066Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10         filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11         active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12         -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20     Run:
21         python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22     """
23     from __future__ import annotations
24
25     import json
26     import argparse
27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score
36     from backend.eval.gemini_refine import merge_overlaps
37
38     Interval = Tuple[float, float]

```

```

39
40     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
41     # Recall-oriented proposal: deliberately permissive so real rallies are among the
42     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
43     PROPOSAL = (0.005, 1.0, 0.5)
44     BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish windowing, no learning
45
46
47     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
48                         proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
49         """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
50         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
51         vt, mg, mwd = proposal
52         wins = TrackNetRunner.trajectory_to_action_windows(
53             points, fps=fps, frame_width=frame_width,
54             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
55         intervals = [(w["start"], w["end"]) for w in wins]
56         gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
57         X: List[List[float]] = []
58         for w, g in zip(wins, gated):
59             dur = max(1e-6, w["end"] - w["start"])
60             win_frames = max(1.0, dur * fps)
61             X.append([
62                 dur, # rally length (sec) ?
63                 float(w["peak_velocity"]), # already normalized by
64                 float(w["mean_velocity"]),
65                 float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
66                 float(g.crossings), # net crossings (count) ?
67                 ])
68         return intervals, X
69
70
71     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
72         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
73         vt, mg, mwd = BASELINE_LOCAL
74         wins = TrackNetRunner.trajectory_to_action_windows(
75             points, fps=fps, frame_width=frame_width,
76             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
77         return [(w["start"], w["end"]) for w in wins]
78
79
80     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
81         fps, fw = float(spec["fps"]), float(spec["frame_width"])
82         intervals, X = build_candidates(spec["trajectory"], fps, fw)
83         gts = load_golden_gts(spec["labels"])
84         y = label_candidates(intervals, gts, iou)
85         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
86                 "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}
87
88
89     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
90                        threshold: float = 0.5) -> List[Interval]:
91         if not intervals:

```

```

92         return []
93     proba = model.predict_proba(X)
94     pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
95     return merge_overlaps(pos, gap_tol=1.0)
96
97
98     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
99             model_out: Optional[str] = None) -> dict:
100         videos = [load_video(s, iou) for s in specs]
101         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
102               f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
103         for v in videos:
104             ceil = score(v["intervals"], v["gts"], iou).recall
105             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
106                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
107
108         result: dict = {"videos": [v["name"] for v in videos]}
109         if len(videos) >= 2:
110             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
111 out) ---")
112             clf, base = [], []
113             for i, held in enumerate(videos):
114                 others = [v for j, v in enumerate(videos) if j != i]
115                 X = [row for v in others for row in v["X"]]
116                 y = [t for v in others for t in v["y"]]
117                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
118                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
119                 sc = score(preds, held["gts"], iou)
120                 bsc = score(held["baseline"], held["gts"], iou)
121                 clf.append(sc)
122                 base.append(bsc)
123                 print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
124 R={sc.recall:.3f} F1={sc.f1:.3f}"
125                       f" | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
126 F1={bsc.f1:.3f}")
127             def mean(xs, a):
128                 return sum(getattr(s, a) for s in xs) / len(xs)
129             print(f"  MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f} "
130                   f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
131                   f"F1={mean(base, 'f1'):.3f}")
132             result["mean_learned_f1"] = mean(clf, "f1")
133             result["mean_baseline_f1"] = mean(base, "f1")
134
135         # Final model on ALL videos (for production / running on a fresh held-out video).
136         Xall = [row for v in videos for row in v["X"]]
137         yall = [t for v in videos for t in v["y"]]
138         final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
139         assert final.w is not None # set by fit()
140         weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
141         print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
142         if model_out:
143             final.save(model_out)
144             print(f"final model (all videos) -> {model_out}")
145             result["model_out"] = model_out
146         return result
147
148     def main() -> None:
149         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
150 classifier.")
151         ap.add_argument("--manifest", required=True,
152                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
153 (labels = Gemini CSV)")
154         ap.add_argument("--iou", type=float, default=0.5)

```


17. user (2026-06-13T19:55:47.178Z)

```
1      """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3      This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4      identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5      no-regression gate, the pinnable classifier artifact, the persisted-candidate
6      substrate) already exists ? this module composes it. No new scoring math lives
7      here**; everything defers to:
8
9      - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`   ? `pct_improvement`
15
16     The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17     (per-frame signals ? run once on the GPU/WSL box, persisted into
18     `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19     signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20     given persisted candidate features it produces `List[Interval]` deterministically,
21     with no GPU and zero production risk. So most experiments are pure offline
22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import json
41     import logging
42     import os
43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54
55     logger = logging.getLogger(__name__)
56
57     # Where a comparison run appends one reproducible record. Tracked location (NOT under
58     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
59     # regression.DEFAULT_BASELINE_PATH.
60     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
61     _METRICS = ("precision", "recall", "f1", "mean_iou")
62
63
```

```

64     class ExperimentError(ValueError):
65         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
66         score an unlabeled video with no pinned model, or a gate referencing an absent
67         feature)."""
68
69
70     # ----- Variant
71
72
73     @dataclass
74     class Variant:
75         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
76
77         A variant turns one video's persisted candidate windows + features into rally
78         intervals. Every knob defaults to the control behaviour (no gate, no learned
79         filter), so a variant that merely carries a new, disabled cue is byte-identical
80         to control ? the core no-regression guarantee.
81
82         Fields:
83         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
84           for the leaderboard and for selecting the served path (the existing
85           ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
86           New cues are recorded here default-OFF.
87         - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
88           ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
89           ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
90           windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
91         - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
92           whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
93           cue persists that feature).
94         - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
95           ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
96           required for ``compare_unlabeled`` on a learned variant.
97         - ``feature_names`` ? the persisted features the learned filter consumes. When set
98           WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
99           (honest) ? the recipe, not a pinned artifact.
100         - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter

```

18. assistant (2026-06-13T19:55:48.583Z)

Let me continue reading the experiment.py file and find PersonDetector:

19. user (2026-06-13T19:55:49.296Z)

```

100         - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
101           overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
102         """
103
104         name: str
105         segmenter: str = "wasb_hybrid"
106         config_overrides: Dict[str, Any] = field(default_factory=dict)
107         cue_flags: Dict[str, bool] = field(default_factory=dict)
108         min_crossings: int = 0
109         min_players: int = 0
110         classifier_model: Optional[str] = None
111         feature_names: Optional[List[str]] = None
112         threshold: float = 0.5
113         merge_gap: float = 1.0
114
115         def is_learned(self) -> bool:
116             """True if this variant applies a learned classifier (pinned model or
recipe)."""
117             return self.classifier_model is not None or bool(self.feature_names)
118

```

```

119     def to_dict(self) -> dict:
120         return {
121             "name": self.name,
122             "segmenter": self.segmenter,
123             "config_overrides": self.config_overrides,
124             "cue_flags": self.cue_flags,
125             "min_crossings": self.min_crossings,
126             "min_players": self.min_players,
127             "classifier_model": self.classifier_model,
128             "feature_names": self.feature_names,
129             "threshold": self.threshold,
130             "merge_gap": self.merge_gap,
131         }
132
133     @classmethod
134     def from_dict(cls, d: dict) -> "Variant":
135         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
136         return cls(**{k: v for k, v in d.items() if k in known})
137
138     def spec_hash(self) -> str:
139         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
140         and makes a result reproducible. The pinned **control** variant always hashes
141         the
142         same, so a regression is always traceable to a recipe change, never to drift."""
143         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
144         return sha1(blob.encode()).hexdigest()[:12]
145
146 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
147 #: filter. Always the comparison reference; "rollback" = make this the served variant.
148 CONTROL = Variant(name="control")
149
150
151 # ----- persisted candidates
152
153
154 @dataclass
155 class VideoCandidates:
156     """One video's persisted detection, ready for offline re-scoring.
157
158     ``intervals`` are the candidate windows; ``features`` is column-major
159     (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
160     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
161     with ``load_video_candidates``, or directly in tests.
162     """
163
164     name: str
165     intervals: List[Interval]
166     features: Dict[str, List[float]] = field(default_factory=dict)
167     gts: Optional[List[Interval]] = None
168
169
170 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
171     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
172     `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
173     col = vc.features.get(name)
174     n = len(vc.intervals)
175     if col is None:
176         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
177                        name, vc.name)
178         return [0.0] * n
179     if len(col) != n:
180         raise ExperimentError(
181             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
182     return [float(x) for x in col]

```

```

183
184
185     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
186         cols = [_feature_column(vc, name) for name in feature_names]
187         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
188
189
190     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
191         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
192         n = len(vc.intervals)
193         mask = [True] * n
194         if variant.min_crossings > 0:
195             col = _feature_column(vc, "net_crossings")
196             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
197         if variant.min_players > 0:
198             col = _feature_column(vc, "players_in_court")
199             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]

```

20. user (2026-06-13T19:55:49.561Z)

backend\pipeline\detectors\person_detector.py

21. user (2026-06-13T19:55:51.363Z)

```

1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7     later the learned classifier reads its persisted features. Keeping detection here (not
in
8     the segmenter) is what makes "add a cue = register a producer" true.
9
10    Behaviour is unchanged from the in-lined version ? only the home moved:
11    - `lazy_load_model`           ? load a permissively-licensed torchvision detector
12    - `make_tracker`             ? a fresh supervision ByteTrack per chunk
13    - `detect_and_track`         ? one frame ? confident persons with persistent IDs
14    - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15    with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16    that feed the candidate `stats` JSON and the learned fusion features.
17
18    The model libs (torchvision, supervision) are imported lazily inside the methods, and
the
19    detector weights load only on first detection (`lazy_load_model`) ? so importing the
20    segmenter registry never loads a detector model, and tests can mock the seams. (torch +
cv2
21    are top-level imports here; CI's import-smoke stubs them, and they're present on the GPU
22    machine.) Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
23    (ADR-005; ultralytics' AGPL YOLO was dropped).
24    """
25    import logging
26    from typing import Any, Dict, List, Optional, Tuple
27
28    import cv2
29    import torch
30
31    from backend.utils.geometry import get_velocity_normalised
32
33    logger = logging.getLogger(__name__)
34
35    # torchvision detection models are trained on COCO where the "person" category is

```

```

36 # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
37 # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
38 _PERSON_CLASS_ID = 1
39
40 # Permissively-licensed torchvision detectors selectable via config
41 # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
42 _DETECTOR_FACTORIES = {
43     "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
44     "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
45     "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
46 }
47 _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
48
49
50 class PersonDetector:
51     """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53     Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54     ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55     min_action_window_duration}``); the model loads lazily on first use.
56     """
57
58     def __init__(self, config: Dict[str, Any]):
59         self.config = config
60         # Dynamic torchvision module (None until lazy_load_model); typed Any so the
61         # `self.model([tensor])` inference call type-checks without a quarantine.
62         self.model: Any = None
63         self.device: Optional[str] = None
64
65     def lazy_load_model(self) -> None:
66         if self.model is not None:
67             return
68
69         # Imported lazily so the module imports without torchvision present and so
70         # the test suite (which pre-sets self.model) never pulls in model weights.
71         from torchvision.models import detection as tv_detection
72
73         idx_cfg = self.config.get("indexing", {})
74         use_gpu = idx_cfg.get("use_gpu", True)
75         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
76         if use_gpu and not torch.cuda.is_available():
77             logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
78
79         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
80         if model_name not in _DETECTOR_FACTORIES:
81             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
82             model_name = _DEFAULT_DETECTOR
83
84         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
85         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
86         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
87         self.model = builder(weights="DEFAULT")
88         self.model.eval()
89         self.model.to(self.device)
90
91     def make_tracker(self, fps: float) -> Any:
92         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
93
94         Imported lazily so tests can mock this method without supervision installed.
95         """
96         import supervision as sv
97         frame_rate = max(1, int(round(fps)))
98         return sv.ByteTrack(frame_rate=frame_rate)

```

```
99
100     def detect_and_track(self, frame: Any, tracker: Any,
```

22. user (2026-06-13T19:55:51.626Z)

```
200         return mask
201
202
203     def predict(variant: Variant, vc: VideoCandidates,
204                 model: Optional[LogisticRegression] = None) -> List[Interval]:
205         """Variant prediction: gate by persisted features, optionally filter by a learned
206         model, then merge. Pure and deterministic.
207
208         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
209         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
210         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
211         is nothing to score with.
212         """
213         mask = _gate_mask(variant, vc)
214         if variant.feature_names:
215             if model is None:
216                 if variant.classifier_model is None:
217                     raise ExperimentError(
218                         f"variant {variant.name!r} is learned but has no model to predict "
219                         f"with (pass a fitted model or set classifier_model)")
220                 model = LogisticRegression.load(variant.classifier_model)
221                 proba = model.predict_proba(_feature_matrix(vc, variant.feature_names))
222                 mask = [m and (float(proba[i]) >= variant.threshold) for i, m in
223                         enumerate(mask)]
224                 kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
225                 return merge_overlaps(kept, gap_tol=variant.merge_gap)
226
227     # ----- variant scoring
228
229
230     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
231     LogisticRegression:
232         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
233         the
234         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
235         X: List[List[float]] = []
236         y: List[int] = []
237         for vc in videos:
238             if vc.gts is None:
239                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
240             X.extend(_feature_matrix(vc, variant.feature_names or []))
241             y.extend(label_candidates(vc.intervals, vc.gts, iou))
242         if not X:
243             raise ExperimentError("cannot train a learned variant on zero candidates")
244         return LogisticRegression().fit(X, y, variant.feature_names)
245
246     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
247                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
248         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
249         mean aggregate.
250
251         Prediction policy:
252         - **static variant** (no learned filter): predict each video directly ? no
253         training,
254         so no leakage; per-video scoring is already honest.
255         - **learned, pinned model** (``classifier_model`` set): score every video with the
256         SHIPPED model. ? optimistic if those videos were in its training set ? use this
257         to
```

```

255         report "how the shipped artifact does here", not for the promotion decision.
256     - **learned recipe** (`feature_names`, no pinned model) + ``honest=True``: LOVO
?
257         train on the OTHER videos, predict the held-out one. The number to trust.
258     - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
259     """
260     for vc in videos:
261         if vc.gts is None:
262             raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
263
264     per_video: List[Dict[str, Any]] = []
265     predictions: Dict[str, List[Interval]] = {}
266
267     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
268     pinned_model = (LogisticRegression.load(variant.classifier_model)
269                     if variant.classifier_model is not None else None)
270     all_model = None
271     if recipe_lovo and not honest:
272         all_model = _train(variant, videos, iou)
273
274     for i, vc in enumerate(videos):
275         if recipe_lovo and honest:
276             others = [v for j, v in enumerate(videos) if j != i]
277             if not others:
278                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
279             model: Optional[LogisticRegression] = _train(variant, others, iou)
280             elif recipe_lovo:                                     # honest=False ? one model over all
videos
281                 model = all_model
282             else:                                                # static variant or pinned model
283                 model = pinned_model
284             preds = predict(variant, vc, model=model)
285             assert vc.gts is not None                            # guarded at function top
286             sc = score(preds, vc.gts, iou)
287             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
288             predictions[vc.name] = preds
289
290     aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
291                  for m in _METRICS}
292     return {
293         "variant": variant.name,
294         "spec_hash": variant.spec_hash(),
295         "is_learned": variant.is_learned(),
296         "honest_lovo": recipe_lovo and honest,
297         "per_video": per_video,
298         "aggregate": aggregate,
299         "predictions": predictions,
300     }
301
302
303     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
304                iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
305         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
306         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
307
308         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
309         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for

```

```

310     `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
311     ""
312     if len(videos) < 1:
313         raise ExperimentError("compare needs at least one labeled video")
314     eval_a = evaluate_variant(videos, variant_a, iou, honest)
315     eval_b = evaluate_variant(videos, variant_b, iou, honest)
316
317     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
318     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
319
320     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
321     per_video_delta = [
322         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
323         for p in eval_b["per_video"]]
324     ]
325
326     # One fingerprint over the whole label set ? detects "the deltas were measured
against
327     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
328     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
329
330     return {
331         "iou": iou,
332         "label_set_hash": gt_hash(all_gts),
333         "num_videos": len(videos),
334         "variant_a": eval_a,
335         "variant_b": eval_b,
336         "delta": delta,
337         "delta_pct": delta_pct,
338         "per_video_delta": per_video_delta,
339     }
340
341
342     # ----- SxS on unlabeled video
343
344
345     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
346         agree_iou: float = 0.5) -> Dict[str, Any]:
347         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
348
349         With no labels there is no lift to measure ? instead this surfaces where the two

```

23. assistant (2026-06-13T19:55:53.042Z)

Now let me search for load_video_candidates:

24. user (2026-06-13T19:55:53.417Z)

Found 1 file
backend\eval\experiment.py

25. user (2026-06-13T19:55:53.899Z)

```

100     def detect_and_track(self, frame: Any, tracker: Any,
101         score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
102         """Detect persons in a frame and assign persistent track IDs.
103
104         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
105         This replaces the old ultralytics ``model.track(...)`` call, which bundled
106         detection AND association: torchvision does the detection, supervision's

```



```

107         ByteTrack does the association.
108         """
109         import supervision as sv
110
111         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
112         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
113         tensor = torch.from_numpy(rgb).permute(2, 0,
114 1).float().div(255.0).to(self.device)
115         with torch.no_grad():
116             outputs = self.model([tensor])
117
118         out = outputs[0]
119         boxes = out["boxes"].detach().cpu().numpy()
120         labels = out["labels"].detach().cpu().numpy()
121         scores = out["scores"].detach().cpu().numpy()
122
123         # Keep only confident person detections.
124         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
125         if not mask.any():
126             return []
127
128         # ByteTrack requires confidence to be populated (it filters on it internally).
129         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
130 class_id=labels[mask])
131         tracked = tracker.update_with_detections(dets)
132
133         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
134         for i in range(len(tracked)):
135             tid = tracked.tracker_id[i]
136             if tid is None:
137                 continue
138             x1, y1, x2, y2 = tracked.xyxy[i]
139             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
140         return result
141
142     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
143 velocity_thresh: float, merge_gap: float,
144 frame_skip: int = 3,
145 chunk_index: Optional[int] = None) ->
146 List[Dict[str, Any]]:
147     """
148     Runs person detection + tracking on a video chunk and extracts high-motion
149     action windows. Returns a list of dicts in the full video's timeframe, each
150     with:
151
152         start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
153         track_id_count, chunk_index.
154
155     Args:
156         frame_skip: Process every Nth frame (1 = every frame). Higher values
157             dramatically reduce inference cost with minimal quality loss.
158         chunk_index: Index of the source chunk (recorded on each window for
159             traceability).
160     """
161     cap = cv2.VideoCapture(chunk_path)
162     if not cap.isOpened():
163         logger.error(f"Could not open {chunk_path}")
164         return []
165
166     fps = cap.get(cv2.CAP_PROP_FPS)
167     if fps <= 0:
168         fps = 30.0
169
170     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
171     if frame_width <= 0:
172         frame_width = 1280.0 # Sensible fallback

```

```

167
168     # Effective sampling rate after frame-skipping
169     effective_fps = fps / max(frame_skip, 1)
170
171     # A fresh tracker per chunk ? track IDs only need to be consistent within a
172     # chunk (windows are computed per chunk, then merged across chunks by time).
173     tracker = self.make_tracker(effective_fps)
174     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
175
176     # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
177     # so windows can be enriched with motion stats for logging + fine-tuning.
178     active_frames = []
179     frame_idx = 0
180     track_history: Dict[int, Tuple[float, float, float, float]] = {}
181
182     while True:
183         ret, frame = cap.read()
184         if not ret:
185             break
186
187         # Skip frames for performance ? at 30fps, every 3rd frame still
188         # gives ~10 samples/second, plenty for human-scale velocity.
189         if frame_idx % max(frame_skip, 1) != 0:
190             frame_idx += 1
191             continue
192
193         # Detect persons + assign persistent track IDs.
194         detections = self.detect_and_track(frame, tracker, score_thresh)
195
196         frame_peak_velocity = 0.0
197         frame_active_ids = set()
198         current_ids = set()
199

```

26. user (2026-06-13T19:55:56.156Z)

```

409-         "aggregate": result["variant_a"]["aggregate"]},
410-         "variant_b": {"name": result["variant_b"]["variant"],
411-                        "spec_hash": result["variant_b"]["spec_hash"],
412-                        "aggregate": result["variant_b"]["aggregate"]},
413-         "delta": result.get("delta"),
414-     }
415-     entries: List[Dict[str, Any]] = []
416-     if os.path.isfile(path):
417-         with open(path, encoding="utf-8") as f:
418-             entries = json.load(f)
419-     entries.append(row)
420-     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
421-     with open(path, "w", encoding="utf-8") as f:
422-         json.dump(entries, f, indent=2)
423-     logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
424-                 row["variant_a"]["name"], row["variant_b"]["name"],
425-                 (result.get("delta") or {}).get("f1", 0.0), path)
426-     return row
427-
428-
429-def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
430-                           gts: Optional[List[Interval]] = None,
431-                           feature_names: Sequence[str] = ()) -> VideoCandidates:
432-     """Bridge persisted detection ? `VideoCandidates` via
433-     `Database.query_candidate_segments`.
434-
435-     ``feature_names`` selects which persisted features to lift out of each row's ``stats``
436-     JSON (``"duration"`` is derived from the window bounds). This is the only DB-touching

```

```

436-     function; the comparison itself is pure and offline. Exercised owner-side on a real DB,
437-     but works against any `Database` (including an in-memory one in tests).
438-     """
439-     rows = db.query_candidate_segments(video_id, detector=detector)
440-     intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
441-     features: Dict[str, List[float]] = {}
442-     for name in feature_names:
443-         if name == "duration":
444-             features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in rows]
445-         else:
446-             features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in rows]
447-     return VideoCandidates(name=video_id, intervals=intervals, features=features, gts=gts)

```

27. assistant (2026-06-13T19:55:57.644Z)

Now let me check for any existing audio-related features or cue framework, and see the structure of stats:

28. user (2026-06-13T19:55:58.483Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-37-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-38-Interval =
Tuple[float, float]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-39-
backend\eval\distill_local.py:40:FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity",
"active_ratio", "net_crossings"]
backend\eval\distill_local.py-41-# Recall-oriented proposal: deliberately permissive so real
rallies are among the
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-42-#
candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-43-PROPOSAL =
(0.005, 1.0, 0.5)
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-113-
others = [v for j, v in enumerate(videos) if j != i]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-114-
X = [row for v in others for row in v["X"]]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-115-
y = [t for v in others for t in v["y"]]
backend\eval\distill_local.py:116:         model = LogisticRegression().fit(X, y,
FEATURE_NAMES)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-117-
preds = predict_rallies(model, held["X"], held["intervals"], threshold)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-118-
sc = score(preds, held["gts"], iou)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-119-
bsc = score(held["baseline"], held["gts"], iou)
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-132-     #
Final model on ALL videos (for production / running on a fresh held-out video).
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-133-     Xall =
[row for v in videos for row in v["X"]]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-134-     yall =
[t for v in videos for t in v["y"]]
backend\eval\distill_local.py:135:         final = LogisticRegression().fit(Xall, yall,
FEATURE_NAMES)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-136-     assert
final.w is not None # set by fit()
backend\eval\distill_local.py:137:         weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for
w in final.w]))
backend\eval\distill_local.py-138-     print(f"\nfinal model standardized weights (sign\size =
influence): {weights}")
backend\eval\distill_local.py-139-     if model_out:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-140-

```

```

final.save(model_out)
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-25-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-26-# YOLO Stage-1
persists these per candidate window in `candidate_segments.stats`
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-27-# (see
HybridYoloSegmenter). Duration is derived from the window bounds.
backend\eval\training.py:28:YOLO_FEATURE_NAMES = [
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-29-     "duration",
"peak_velocity", "mean_velocity", "active_frame_count", "track_id_count",
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-30-]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-31-
--
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_training.py-11-
tests\test_eval_training.py-12-def test_extract_yolo_features_shape_and_values():
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_training.py-13-     feats =
training.extract_yolo_features(_row(10, 25, peak=0.4, mean=0.2, frames=30, tracks=2))
tests\test_eval_training.py:14:     assert len(feats) == len(training.YOLO_FEATURE_NAMES)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_training.py-15-     assert
feats == [15.0, 0.4, 0.2, 30.0, 2.0]
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_training.py-16-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_training.py-17-
--
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_training.py-53-     X, y,
intervals = training.build_training_set(rows, gt, iou_threshold=0.5)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_training.py-54-     assert y
== [1, 0]
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_training.py-55-     assert
intervals == [(10.0, 20.0), (500.0, 505.0)]
tests\test_eval_training.py:56:     assert len(X) == 2 and len(X[0]) ==
len(training.YOLO_FEATURE_NAMES)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_training.py-57-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_training.py-58-
tests\test_eval_training.py-59-def test_build_training_set_custom_feature_fn():
--
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_experiment.py-216-     db =
Database(str(tmp_path / "t.db"))
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_experiment.py-217-
db.add_video("vid1", "/x.mp4", "ok", [])
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_experiment.py-218-
db.add_candidate_segment("vid1", "wasb_hybrid", 0.0, 10.0,
tests\test_experiment.py:219:     stats={"net_crossings": 4,
"peak_velocity": 0.3})
tests\test_experiment.py:220:     db.add_candidate_segment("vid1", "wasb_hybrid", 12.0, 13.0,
stats={"net_crossings": 0})
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_experiment.py-221-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_experiment.py-222-     vc =
load_video_candidates(db, "vid1", detector="wasb_hybrid", gts=[(0.0, 10.0)],
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_experiment.py-223-
feature_names=["net_crossings", "duration"])
--
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-8-from types
import SimpleNamespace
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-9-from
unittest.mock import MagicMock, patch
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-10-
tests\test_fusion_hybrid.py:11:from backend.pipeline.detectors.fusion_features import
PERSON_FEATURE_NAMES
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-12-from
backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-13-
tests\test_fusion_hybrid.py-14-# Window A [1,4]s: shuttle alternates across the net every frame
-> many crossings (a rally).
--
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-67-     assert

```

```

len(stats) == 2                                # both windows persisted (superset)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-68-   assert
all("net_crossings" in s for s in stats)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-69-   # Window
A (rally) crosses many times; window B (held) zero.
tests\test_fusion_hybrid.py:70:   assert stats[0]["net_crossings"] >= 2
tests\test_fusion_hybrid.py:71:   assert stats[1]["net_crossings"] == 0.0
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-72-   # Person
cue OFF by default -> NONE of the person features present.
tests\test_fusion_hybrid.py:73:   assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for
s in stats)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-74-   # The 4
base motion keys are still there.
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-75-   assert
all({"peak_velocity", "mean_velocity", "active_frame_count", "track_id_count"} <= set(s)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-76-   for s in stats)
--
tests\test_fusion_hybrid.py-110-   with
patch("backend.pipeline.detectors.person_detector.PersonDetector", return_value=fake_pd):
tests\test_fusion_hybrid.py-111-   db, _provider, _res = _run(indexing={"fusion":
{"cue_flags": {"person": True}}})
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-112-   stats =
_stats_calls(db)
tests\test_fusion_hybrid.py:113:   assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s
in stats)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-114-   assert
all("net_crossings" in s for s in stats)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-115-   assert
stats[0]["players_in_court"] == 2.0           # 2 in-court players every sampled frame
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_hybrid.py-116-   assert
stats[0]["mean_player_motion"] > 0.0          # both tracks moved -> non-zero motion
--
tests\test_distill_local.py-21-def test_build_candidates_feature_shape(tmp_path):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_distill_local.py-22-
intervals, X = dl.build_candidates(_write_traj(tmp_path / "t.csv"), fps=10.0, frame_width=100.0)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_distill_local.py-23-   assert
len(intervals) == len(X) >= 1
tests\test_distill_local.py:24:   assert all(len(row) == len(dl.FEATURE_NAMES) for row in X)
# 5 invariant features
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_distill_local.py-25-   assert
all(d > 0 for d, *_ in X)                     # duration positive
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_distill_local.py-26-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_distill_local.py-27-
tests\test_distill_local.py-28-def test_predict_rallies_respects_threshold(tmp_path):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_distill_local.py-29-
intervals, X = dl.build_candidates(_write_traj(tmp_path / "t.csv"), fps=10.0, frame_width=100.0)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_distill_local.py-30-   y = [1] +
[0] * (len(intervals) - 1)
tests\test_distill_local.py:31:   model = LogisticRegression(epochs=100).fit(X, y,
dl.FEATURE_NAMES)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_distill_local.py-32-   none =
dl.predict_rallies(model, X, intervals, threshold=1.01) # impossible cutoff -> nothing
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_distill_local.py-33-   assert
none == []
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_distill_local.py-34-   some =
dl.predict_rallies(model, X, intervals, threshold=0.0) # everything passes -> merged windows
--
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_features.py-135-
shuttle = [_shuttle(0, 20, 25, True)]
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_features.py-136-   feats
= ff.compute_person_features(per_frame, shuttle, 100, 100,
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_features.py-137-   court_polygon=None, net=("y", 0.5))
tests\test_fusion_features.py:138:   assert set(feats) == set(ff.PERSON_FEATURE_NAMES)

```

```
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_features.py-139- assert
all(isinstance(v, float) for v in feats.values())
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_features.py-140- assert
feats["two_sided_motion"] == pytest.approx(1.0) # both sides occupied every frame
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_features.py-141-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_features.py-142-
tests\test_fusion_features.py-143-def test_compute_person_features_empty():
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_features.py-144- feats
= ff.compute_person_features({}, [], 100, 100)
tests\test_fusion_features.py:145: assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_fusion_features.py-146- assert
all(v == 0.0 for v in feats.values())
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-190-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-191-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-192-# The ~5 person features persisted
into candidate `stats` (order = the learned-fusion
backend\pipeline\detectors\fusion_features.py:193:# feature columns appended after the shuttle
features in distill_local.FEATURE_NAMES).
backend\pipeline\detectors\fusion_features.py:194:PERSON_FEATURE_NAMES = [
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-195- "players_in_court",
"two_sided_motion", "mean_player_motion",
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-196- "in_court_ratio",
"shuttle_player_colocation",
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-197-]
--
backend\pipeline\detectors\fusion_features.py-203- net: Optional[Net]
= None,
backend\pipeline\detectors\fusion_features.py-204- min_players: int =
2) -> Dict[str, float]:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-205- """Aggregate the per-window person
features from per-frame detections + the in-window
backend\pipeline\detectors\fusion_features.py:206: shuttle trajectory. Pure; returns exactly
`PERSON_FEATURE_NAMES`, all floats."""
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-207- counts =
in_court_counts(per_frame, frame_width, frame_height, court_polygon)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-208- return {
backend\pipeline\detectors\fusion_features.py-209- "players_in_court":
players_in_court(counts),
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\fusion_hybrid.py-82- # over the whole trajectory by
rally_gate (camera-agnostic); reused across windows.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\fusion_hybrid.py-83- gated =
rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\fusion_hybrid.py-84- min_crossings=0, estimate=self._axis_estimate(points))
backend\pipeline\segmenters\fusion_hybrid.py:85: stats["net_crossings"] =
float(gated[0].crossings) if gated else 0.0
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\fusion_hybrid.py-86-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\fusion_hybrid.py-87- fcfg = idx_cfg.get("fusion", {})
backend\pipeline\segmenters\fusion_hybrid.py-88- if fcfg.get("cue_flags",
```

```

{}).get("person", False):
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\fusion_hybrid.py-121-      # If we can't trust the frame
dims, every normalised feature would silently collapse
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\fusion_hybrid.py-122-      # to ~0 ? return an explicit
all-zero person-feature row instead of a half-baked one.
backend\pipeline\segmenters\fusion_hybrid.py-123-      if fw <= 0 or fh <= 0:
backend\pipeline\segmenters\fusion_hybrid.py:124:          return {k: 0.0 for k in
ff.PERSON_FEATURE_NAMES}
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\fusion_hybrid.py-125-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\fusion_hybrid.py-126-      poly =
fcfg.get("court_polygon")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\fusion_hybrid.py-127-      polygon = [(float(p[0]),
float(p[1])) for p in poly] if poly else None
--
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_trajectory_hybrid_equivalence.py-
105-@pytest.mark.parametrize("cls,family,detector", TWINS)
tests\test_trajectory_hybrid_equivalence.py-106-def
test_persisted_stats_are_exactly_the_four_motion_keys(cls, family, detector):
tests\test_trajectory_hybrid_equivalence.py-107-    """The fusion-P2 enrichment seam must leave
the twins' persisted stats byte-identical:
tests\test_trajectory_hybrid_equivalence.py:108:    the base _enrich_candidate_stats is
identity, so NO net_crossings / person keys leak in."""
tests\test_trajectory_hybrid_equivalence.py-109-    db, _res = _run(cls, [{"start_time": 0.5,
"end_time": 3.0}])
C:\Users\avidu\Projects\badminton-highlight-
indexer\tests\test_trajectory_hybrid_equivalence.py-110-    _, kwargs =
db.add_candidate_segment.call_args
C:\Users\avidu\Projects\badminton-highlight-
indexer\tests\test_trajectory_hybrid_equivalence.py-111-    assert set(kwargs["stats"]) == {

```

29. user (2026-06-13T19:55:58.574Z)

backend\pipeline\detectors\rally_gate.py

30. assistant (2026-06-13T19:56:00.150Z)

Perfect. Now let me check the structure of how fusion_features works and confirm the wiring pattern:

31. user (2026-06-13T19:56:00.881Z)

```

100     def detect_and_track(self, frame: Any, tracker: Any,
101                          score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
102         """Detect persons in a frame and assign persistent track IDs.
103
104         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
105         This replaces the old ultralytics ``model.track(...)`` call, which bundled
106         detection AND association: torchvision does the detection, supervision's
107         ByteTrack does the association.
108         """
109         import supervision as sv
110
111         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
112         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
113         tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
114         with torch.no_grad():

```

```

115         outputs = self.model([tensor])
116
117         out = outputs[0]
118         boxes = out["boxes"].detach().cpu().numpy()
119         labels = out["labels"].detach().cpu().numpy()
120         scores = out["scores"].detach().cpu().numpy()
121
122         # Keep only confident person detections.
123         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
124         if not mask.any():
125             return []
126
127         # ByteTrack requires confidence to be populated (it filters on it internally).
128         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
129         tracked = tracker.update_with_detections(dets)
130
131         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
132         for i in range(len(tracked)):
133             tid = tracked.tracker_id[i]
134             if tid is None:
135                 continue
136             x1, y1, x2, y2 = tracked.xyxy[i]
137             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
138         return result
139
140     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
141                                         velocity_thresh: float, merge_gap: float,
142                                         frame_skip: int = 3,
143                                         chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
144         """
145         Runs person detection + tracking on a video chunk and extracts high-motion
146         action windows. Returns a list of dicts in the full video's timeframe, each
with:
147             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
148             track_id_count, chunk_index.
149

```

32. user (2026-06-13T19:56:00.930Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple

```



```

25
26 import cv2
27
28 from backend.pipeline.segmenters.base import BasePlaySegmenter
29 from backend.utils.video import get_video_duration, extract_video_chunk
30 from backend.sports import sport_registry
31 from backend.providers import provider_registry
32 from backend.pipeline.detectors.base import DetectorRunner
33
34 logger = logging.getLogger(__name__)
35
36
37 def _fmt_ts(seconds: float) -> str:
38     seconds = max(0, int(seconds))
39     h, rem = divmod(seconds, 3600)
40     m, s = divmod(rem, 60)
41     return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44 class TrajectoryHybridSegmenter(BasePlaySegmenter):
45     """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46 boundaries."""
47
48     #: config section under `indexing` (velocity_threshold lives there), the
49     #: output subdir for trajectories, and the metadata detector-tag prefix.
50     family: str = ""
51     #: human-readable label used in log lines.
52     display_label: str = ""
53
54     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55 Any]]:
56         """Return (runner, detector-specific detection_params extras)."""
57         raise NotImplementedError
58
59     @staticmethod
60     def _probe_fps_width(video_path: str) -> tuple:
61         """Return (fps, frame_width) for a video, with sensible fallbacks."""
62         cap = cv2.VideoCapture(video_path)
63         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65         cap.release()
66         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68     @staticmethod
69     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70         """Provider-reported exchange count, else a duration-based estimate.
71
72         One canonical implementation ? the twins had drifted (wasb relied on
73         ``int(None)`` raising into the fallback; same result, by accident).
74         """
75         shot_count = segment.get("shuttle_exchange_count")
76         if shot_count is None:
77             return max(3, int(duration / 0.8))
78         try:
79             return int(shot_count)
80         except (ValueError, TypeError):
81             return max(3, int(duration / 0.8))
82
83     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84     -----
85     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                                frame_width: float, video_path: str,
87                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89
90 BASE

```

```

86         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
87         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
88         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
89         return {
90             "peak_velocity": cw["peak_velocity"],
91             "mean_velocity": cw["mean_velocity"],
92             "active_frame_count": cw["active_frame_count"],
93             "track_id_count": cw["track_id_count"],
94         }
95
96     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                           window_stats: List[Dict[str, Any]],
98                           idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);

```

33. assistant (2026-06-13T19:56:03.323Z)

Excellent. Now I have all the context I need. Let me create a comprehensive design document:

34. user (2026-06-13T19:56:44.883Z)

Structured output provided successfully

35. assistant (2026-06-13T19:56:53.255Z)

Perfect. I've completed the design specification for wiring the audio hit detector as a cue/feature. Here's what I've documented:

Key Findings:

****Audio Hit Detector Status:****

- The classical onset-envelope detector (`hit\_detector.py`, lines 23-146) is ****already on master****
- E1 probe (2026-06-07) found ~2.2x discrimination ceiling (in-rally vs dead-time); band-pass didn't help
- Decision: ****persist the weak signal, let the learned scorer decide**** (honest approach)

****Integration Architecture (P4):****

1. ****New Audio Features Producer**** (`audio\_features.py`):
 - 3 features per window: `audio\_hit\_count`, `audio\_hit\_rate`, `audio\_hit\_strength\_mean`
 - Pure Python, GPU-free, mirrors `fusion\_features.py` discipline
 - Gracefully degrades to 0.0 on empty input
2. ****FusionSegmenter Wiring****:
 - Extends `\_enrich\_candidate\_stats()` with audio cue branch (same pattern as person cue)
 - Lines 76-90 in `fusion\_hybrid.py` ? add audio branch with cond `fcfg.get("cue\_flags", {})).get("audio", False)`
 - New private method `\_audio\_features()` lazily caches hits once per video, computes features per window
3. ****Persisted Features Path****:
 - Into `candidate\_segments.stats` JSON (existing mechanism)
 - Only when `fusion.cue\_flags.audio = true` (default-OFF)
 - Ordered after net_crossings (existing) and person features (if enabled)
4. ****Experiment Harness Integration****:
 - `Variant` dataclass already supports `cue\_flags` and arbitrary `feature\_names` (no changes needed)
 - Audio is a **variant**, not a code branch
 - Offline A/B via `compare()` using LOVO-honest training

- ``load_video_candidates()`` already maps missing columns ? 0.0 (backwards compatible)

5. ****Config & Rollback****:

- Under ``indexing.fusion``: ``cue_flags.audio``, ``audio_params`` (threshold_k, band, sr, min_gap_sec)
- Default-OFF ? byte-identical to current behavior
- Rollback: flip flag, no code changes

****Honest Assessment per E1:****

- Weak signal (~2.2× ceiling) is unlikely to produce meaningful weights
- But framework accepts it without forcing adoption; let the golden-label corpus + learned scorer decide
- Zero downside risk; plumbing ready if E2 (learned timbre classifier) lands